Project Work Week 1:

Remember, you have to perform work on each topic and on **each** bullet point. Be done with the project work before we meet again next week.

1) Bureaucracy

- Register your groups in teams

2) Process

- Form groups of **five** students, if you are lacking team members, find each other in the first exercise session or use our learnit forum.
- Create an organization on GitHub
  - The organization has to be named `ITU-BDSA2026-GROUPno`, where `no` is replaced with your group number
  - Add the five members of your group to that organization
- Within the organization, create a new public repository called `Bison`.
- All the project work of the next weeks takes place in this repository.

3) Software Development

During the first four weeks of this course, you will create and gradually enhance a first version of a Bison application. It will be a command-line application that will be able to display observations from and to store observations in a local comma-separated-value (CSV) file. For now, we will not distinguish between observations and comments in the discussion, and just allow people to record what they see. For this weeks project work do the following:

1. Create a .NET CLI App called `Bison.CLI` (You may create a new one or reuse your in-class work)
2. The comma-separated-value (CSV) file `bison_observe_cli_db.csv` is a basic database containing a list of observations, their authors, and time of creation.
3. Make your `Bison.CLI` display all cheeps that are stored in the file by writing them line-wise to stdout (i.e., `Console.WriteLine()`)

- When your program is called from the command line (either with `Bison.CLI read` or `dotnet run -- read`) the output should look as in the following:

```
YYY @ 08/11/26 14:09:20: A bird at DR Byen
XXX @ 08/12/26 14:19:38: I think that is a Heron at DR Byen
ZZZ @ 08/22/26 15:04:47: Ardea cinerea at DR Byen
```

4. Once display of observation from CSV files works, add a second option to your program so that you can store observations in the CSV file.

- Your program shall be called like `Bison.CLI observe "Heron at DR Byen"` (or similarly via `dotnet run -- observe "Heron at DR Byen"`)
- The observation `"Heron at DR Byen"` shall be appended to the CSV file.
- The author shall be set to the name of the currently logged in user, i.e., the user name from the operating system.
- The time stamp shall be set to the current Unix time stamp

Note, while developing your program continuously log its evolution via commits, e.g., on commit per item above, and push your work to your remote repository on GitHub. Remember to register everybody who contributed to a commit as co-authors (via `Co-authored-by:` lines in the commit messages, see lecture slides.)

Design considerations:

Make this week's version as simple as possible. That is, apply the KISS design principle, which suggest that *"simplicity should be a key goal in design, and unnecessary complexity should be avoided"*. Hence, likely all code is organized in `Program.cs`.

Hints:

- You can create a CLI tool in .NET using this guide.
  - Per default, the project template create a *new style* `Program.cs` file, read more about differences to the *old style* here
- How-to read lines from a text file in C#?
  - When searching the internet on how to parse comma-separated-values from a string in C#, you will likely find answers like this one, which relies on a regular expression.
  - In case you use regular expressions for parsing, read the corresponding documentation.
- How-to append lines to text files in C#?
- You can receive current user names from the operating system via the `Environment.UserName` property
- The following documentation pages might help you with storing time stamps properly:
  - https://learn.microsoft.com/en-us/dotnet/api/system.datetimeoffset.utcnow?view=net-8.0
  - https://learn.microsoft.com/en-us/dotnet/api/system.datetimeoffset.tounixtimeseconds?view=net-8.0

4) Ethics

The following bullet points hold true for the entire project work during this semester (also for other subjects).

- **Do not** copy source code, documentation, configuration, or any other

artifact from other project groups without asking for their permission. This holds for the remainder of the term (and for the remainder of your career).

- **Do not** copy source code, documentation, configuration, or any other artifact from any other sources without being sure that you are allowed to do so.
  - In case you are allowed, then cite the sources of respective artifacts explicitly in your repository by providing links back to the original.
  - The back links have to appear in a separate text file in your repository or directly in the sources as comments.
- In case you are using Large Language Models (LLMs), like ChatGPT, GitHub CodePilot, etc. while creating your solution:
  - Register the respective tool as co-author in the corresponding commits.
  - Make sure that you actually understand what these tools generate and if it is really what you would like to implement as a solution.
  - Make sure that what you receive from these tools is actually .NET 8 code and not code generated for previous versions of the framework.
  - Record why you decided to use LLMs, and how you validated their output, i.e., made sure their output is what you indeed intended.